

ETL, ELT and Incremental Data Pipelines

Document ID	DOC010
Project	Incremental Pipelines
Category	Data Engineering
Batch	batch_3

Purpose: Original educational notes created as a searchable PDF corpus for a portfolio-scale incremental RAG pipeline. The material is designed to contain meaningful technical sections that naturally produce multiple retrieval chunks.

Full and Incremental Loads

A full load processes the complete available dataset. It is simple and can be useful for initial ingestion or small sources, but repeated full processing becomes wasteful as data grows. An incremental load identifies only records or files that are new or changed since a previous successful boundary.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Watermarks

A watermark represents processed progress, often using a timestamp or increasing numeric key. The pipeline reads the previous watermark, determines a new upper boundary, processes records within the interval, and updates the stored watermark only after successful completion. This prevents a failed run from advancing progress past unprocessed data.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

File-Based Incremental Processing

For file ingestion, a processed-file registry can record file name, document identifier, hash, timestamp, and status. New files are processed, already successful unchanged files are skipped, and failed files can be retried. Content hashes can detect a changed file even when its name remains the same.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Upserts and Merge Logic

When incremental data can update existing entities, the target may require an upsert rather than append-only loading. Merge logic matches incoming rows to target keys, updates existing records when required, and inserts new records. The business key and update rules must be explicit to avoid duplicate or incorrectly overwritten data.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Audit Logging

A useful batch audit record can include batch ID, received count, processed count, skipped count, records or chunks created, processing timestamp, duration, and status. Audit logging provides evidence of incremental behavior and helps diagnose unexpected counts. It does not need to become a full observability platform to be useful.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

Reruns and Idempotency

Failures and reruns are normal pipeline scenarios. Idempotent design ensures that rerunning a successful batch does not create duplicate output. For a document RAG pipeline, checking document identifiers and content hashes before embedding prevents repeated API calls and duplicate vector entries. The same principle applies broadly to data pipelines.

Practical note. For data engineering work, the design should make processing boundaries explicit, preserve traceability, and avoid unnecessary recomputation when only a subset of the source has changed.

End of document.